

Technical Requirement Document

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

TR-DLT-001: Load Customer and Order Data into Delta Tables

Related Functional Requirement(s): FR-DLT-001

Objective: Implement a PySpark job to load customer and order data from CSV files into Delta tables.

Target Cluster Configuration:

- Cluster Name: Data Engineering Cluster
- Databricks Runtime Version: 10.4 LTS
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 (autoscaling)
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: delta-lake, spark-csv

Source Data Details:

- Dataset Name: Customer Data
- Location: `/Volumes/gen\_ai\_poc\_databrickscOE/sdlc\_wizard/customerdata`
- Format: CSV
- Description: Customer data in CSV format
- Dataset Name: Order Data
- Location: `/Volumes/gen\_ai\_poc\_databrickscOE/sdlc\_wizard/orderdata`
- Format: CSV
- Description: Order data in CSV format

Target Data Details:

- Output Name: Customer Delta Table
- Location: `customer`
- Format: Delta
- Description: Customer data in Delta format
- Output Name: Order Delta Table

- Location: `order`
- Format: Delta
- Description: Order data in Delta format

Job Flow / Pipeline Stages:

- Read customer CSV data into a Delta table
- Read order CSV data into a Delta table
- Add a new column "TotalAmount" to the "order" Delta table

Data Transformations / Business Logic:

- Step: Read CSV data into Delta tables
- Description: Read customer and order CSV data into Delta tables
- Transformation Logic: Use `spark.read.csv()` to read CSV data and `write.format("delta").saveAsTable()` to save as Delta tables
- Step: Add "TotalAmount" column to "order" Delta table
- Description: Calculate "TotalAmount" as the product of "PricePerUnit" and "Qty" columns
- Transformation Logic: Use `withColumn()` to add a new column "TotalAmount" with the calculated value

Error Handling and Logging:

- Log errors and exceptions to a logging table
- Handle null and duplicate records by removing them in subsequent steps

TR-DLT-002: Clean and Process Customer and Order Data

Related Functional Requirement(s): FR-DLT-002

Objective: Implement data cleaning and processing for customer and order data.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details: Same as TR-DLT-001 (Customer and Order Delta tables)

Target Data Details: Same as TR-DLT-001 (Customer and Order Delta tables)

Job Flow / Pipeline Stages:

- Remove rows with null values from the "customer" and "order" Delta tables
- Remove duplicate rows from the "customer" and "order" Delta tables

Data Transformations / Business Logic:

- Step: Remove null values
- Description: Remove rows with null values from the "customer" and "order" Delta tables
- Transformation Logic: Use `dropna()` to remove rows with null values
- Step: Remove duplicates

- Description: Remove duplicate rows from the "customer" and "order" Delta tables
- Transformation Logic: Use ``dropDuplicates()`` to remove duplicate rows

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-003: Create and Load Order Summary Table

Related Functional Requirement(s): FR-DLT-003

Objective: Implement a PySpark job to create and load the "ordersummary" table.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details: Customer and Order Delta tables

Target Data Details:

- Output Name: Order Summary Table
- Location: ``ordersummary``
- Format: Delta
- Description: Joined customer and order data in Delta format

Job Flow / Pipeline Stages:

- Create the "ordersummary" table if it does not exist
- Join the "customer" and "order" Delta tables on the "CustId" column
- Load the joined data into the "ordersummary" table using SCD type 2

Data Transformations / Business Logic:

- Step: Join customer and order data
- Description: Join the "customer" and "order" Delta tables on the "CustId" column
- Transformation Logic: Use ``join()`` to join the two tables
- Step: Load joined data into "ordersummary" table
- Description: Load the joined data into the "ordersummary" table using SCD type 2
- Transformation Logic: Use ``write.format("delta").option("mergeSchema", "true").saveAsTable()`` to load data into the "ordersummary" table

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-004: Update Order Summary Table on Customer Change

Related Functional Requirement(s): FR-DLT-004

Objective: Implement a PySpark job to update the "ordersummary" table when there is a change in the customer table.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details: Customer Delta table and Order Summary table

Target Data Details: Order Summary table

Job Flow / Pipeline Stages:

- Identify changes in the "customer" table
- Update the "ordersummary" table by making old records inactive and new records active
- Update the "StartDate" and "EndDate" columns accordingly to maintain history

Data Transformations / Business Logic:

- Step: Identify changes in customer data
- Description: Identify changes in the "customer" table
- Transformation Logic: Use `exceptAll()` to identify changes in the "customer" table
- Step: Update "ordersummary" table
- Description: Update the "ordersummary" table by making old records inactive and new records active
- Transformation Logic: Use `merge()` to update the "ordersummary" table

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-005: Create and Load Customer Aggregate Spend Table

Related Functional Requirement(s): FR-DLT-005

Objective: Implement a PySpark job to create and load the "customeraggregatespend" table.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details: Order Summary table

Target Data Details:

- Output Name: Customer Aggregate Spend Table
- Location: `customeraggregatespend`
- Format: Delta
- Description: Aggregated customer spend data in Delta format

Job Flow / Pipeline Stages:

- Create the "customeraggregatespend" table if it does not exist
- Aggregate the "TotalAmount" column from the "ordersummary" table by "Name" and "Date" columns
- Load the aggregated data into the "customeraggregatespend" table

Data Transformations / Business Logic:

- Step: Aggregate customer spend data

- Description: Aggregate the "TotalAmount" column from the "ordersummary" table by "Name" and "Date" columns
- Transformation Logic: Use `groupBy()` and `sum()` to aggregate the data
- Step: Load aggregated data into "customeraggregatespend" table
- Description: Load the aggregated data into the "customeraggregatespend" table
- Transformation Logic: Use `write.format("delta").saveAsTable()` to load data into the "customeraggregatespend" table

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-006: Implement Delta Live Tables

Related Functional Requirement(s): FR-DLT-006

Objective: Implement Delta Live Tables to perform the above steps.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details: Same as TR-DLT-001

Target Data Details: Same as TR-DLT-001

Job Flow / Pipeline Stages:

- Implement the data pipelines using Delta Live Tables to perform the above steps

Data Transformations / Business Logic: Same as TR-DLT-001 to TR-DLT-005

Error Handling and Logging: Same as TR-DLT-001